dplyr cheat sheet

MC 451 / MC 501 / MC 427 · the verbs that reshape a table

dplyr is the set of verbs you use to pick rows and columns, make new columns, count things, and join tables. Each entry says what the verb does in one line, shows it on the course Twitch data, and shows the exact output R prints. The last two lines of each entry, **On your data**, rewrite the call with plain column names you can copy, and **If it breaks**, name the error you are most likely to see.

Read the pipe `%>%` as "and then." It hands the table on its left into the function on its right, so a stack of verbs reads top to bottom as the steps you took. Use `%>%`, not `|>`.

Every worksheet starts with these lines, so the examples below assume them:

```
library(tidyverse)   # dplyr comes with the tidyverse
library(v2v)          # the course package and its Twitch data
options(scipen = 10)  # long numbers print in full
```

The data these examples use

`msgs` is one row per chat message, trimmed to six columns so it fits on the page. (`viewers` is `NA` for the first messages, which arrived before the first stream snapshot was recorded.)

```
msgs %>% slice_head(n = 5)
```

```
# A tibble: 5 × 6
  channel    channel_type viewer_band viewers message_length  hour
  <chr>      <fct>        <fct>         <int>          <int> <int>
1 sodapoppin gaming       large            NA             23    21
2 xqcow      non-gaming   medium           NA            441    21
3 forsen     gaming       medium           NA              8    21
4 sodapoppin gaming       large            NA             12    21
5 xqcow      non-gaming   medium           NA            206    21
```

Look at the table

`glimpse()`: see every column at once

Prints one line per column with its type and first few values. The fastest way to remember what is in a table.

```
glimpse(msgs)
```

```
Rows: 35,267
Columns: 6
$ channel        <chr> "sodapoppin", "xqcow", "forsen", "sodapoppin", "xqc…
$ channel_type   <fct> gaming, non-gaming, gaming, gaming, non-gaming, gam…
$ viewer_band    <fct> large, medium, medium, large, medium, medium, large…
$ viewers        <int> NA, NA, NA, NA, NA, 15300, 28076, 4703, 19239, 2855…
$ message_length <int> 23, 441, 8, 12, 206, 37, 6, 7, 4, 7, 3, 4, 12, 21, …
$ hour           <int> 21, 21, 21, 21, 21, 21, 21, 21, 21, 22, 22, 22, 22,…
```

On your data: `glimpse(your_data)`

If it breaks: `object 'your_data' not found` means the table does not exist yet. Run the line that creates it first.

Pick rows and columns

`select()`: keep the columns you name

Keeps only the columns you list, in the order you list them.

```
msgs %>%
  select(channel, channel_type, message_length) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 3
  channel    channel_type message_length
  <chr>      <fct>                <int>
1 sodapoppin gaming                  23
2 xqcow      non-gaming             441
3 forsen     gaming                   8
4 sodapoppin gaming                  12
5 xqcow      non-gaming             206
```

On your data: `your_data %>% select(col_a, col_b)`

If it breaks: `Column 'messege_length' doesn't exist` means a typo in a name. Column names must match exactly, capitals included.

`rename()`: give a column a new name

Renames a column; the new name goes on the left of the =.

```
msgs %>%
  rename(chars = message_length) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 6
  channel    channel_type viewer_band viewers chars  hour
  <chr>      <fct>        <fct>         <int> <int> <int>
1 sodapoppin gaming       large            NA    23    21
2 xqcow      non-gaming   medium           NA   441    21
3 forsen     gaming       medium           NA     8    21
4 sodapoppin gaming       large            NA    12    21
5 xqcow      non-gaming   medium           NA   206    21
```

On your data: `your_data %>% rename(new_name = old_name)`

If it breaks: getting the sides backwards (`old_name = new_name`) gives `object 'old_name' not found`. The name that already exists goes on the right.

`filter()`: keep the rows that pass a test

Keeps only the rows where the test is true. Use `>`, `<`, `==` (equal to), and `!=` (not equal).

```
msgs %>%
  filter(message_length > 200) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 6
  channel    channel_type viewer_band viewers message_length  hour
  <chr>      <fct>        <fct>         <int>          <int> <int>
1 xqcow      non-gaming   medium           NA            441    21
2 xqcow      non-gaming   medium           NA            206    21
3 xqcow      non-gaming   medium        19201            209    22
4 sodapoppin gaming       large         40266            251    22
5 xqcow      non-gaming   medium        25494            287    22
```

On your data: `your_data %>% filter(col_a > 10)`, or `filter(col_a == "yes")` for text.

If it breaks: one `=` where you meant `==` gives `We detected a named input`. Use `==` to test for equality; a single `=` assigns.

`arrange()`: sort the rows

Sorts the rows by a column, smallest first. Wrap the column in `desc()` for largest first.

```
msgs %>%
  arrange(desc(message_length)) %>%
  select(channel, message_length) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  channel          message_length
  <chr>                     <int>
1 xqcow                       501
2 xqcow                       500
3 seucreysonreborn            500
4 seucreysonreborn            500
5 seucreysonreborn            500
```

On your data: `your_data %>% arrange(desc(col_a))`

If it breaks: nothing errors if you forget `desc()`; you just get smallest first. Check the top row is the end you wanted.

`slice_head()`: keep the first few rows

Keeps the first `n` rows. Used all over this sheet to show a short preview.

```
msgs %>%
  slice_head(n = 3)
```

```
# A tibble: 3 × 6
  channel    channel_type viewer_band viewers message_length  hour
  <chr>      <fct>        <fct>         <int>          <int> <int>
1 sodapoppin gaming       large            NA             23    21
2 xqcow      non-gaming   medium           NA            441    21
3 forsen     gaming       medium           NA              8    21
```

On your data: `your_data %>% slice_head(n = 10)`

If it breaks: nothing to break. For a random few instead of the first few, use `slice_sample()` below.

`slice_sample()`: draw a random few rows

Draws `n` rows at random. Put `set.seed()` on the line before it so everyone draws the same rows.

```
set.seed(20261113)
msgs %>%
  slice_sample(n = 3)
```

```
# A tibble: 3 × 6
  channel      channel_type viewer_band viewers message_length  hour
  <chr>        <fct>        <fct>         <int>          <int> <int>
1 superliga    gaming       small          4742             35    16
2 cuppcaake    gaming       small          1194            178    22
3 zeppelinsoul gaming       small          1218             13    18
```

On your data: `set.seed(123); your_data %>% slice_sample(n = 30)`

If it breaks: your rows differ from a classmate's when you skip `set.seed()`, or set a different seed. Same seed, same draw.

Make and change columns

`mutate()`: add or change a column

Adds a new column computed from the others, or overwrites one. Everything else stays.

```
msgs %>%
  mutate(is_long = message_length > 50) %>%
  select(channel, message_length, is_long) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 3
  channel    message_length is_long
  <chr>               <int> <lgl>
1 sodapoppin             23 FALSE
2 xqcow                 441 TRUE
3 forsen                  8 FALSE
4 sodapoppin             12 FALSE
5 xqcow                 206 TRUE
```

On your data: `your_data %>% mutate(new_col = col_a * 2)`

If it breaks: `object 'message_lenght' not found` inside `mutate()` is a typo in the column you are reading.

`if_else()`: pick one of two values per row

Inside a `mutate()`, gives the first value where the test is true and the second where it is false.

```
msgs %>%
  mutate(size = if_else(message_length > 50, "long", "short")) %>%
  select(channel, message_length, size) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 3
  channel    message_length size
  <chr>               <int> <chr>
1 sodapoppin             23 short
2 xqcow                 441 long
3 forsen                  8 short
4 sodapoppin             12 short
5 xqcow                 206 long
```

On your data: `mutate(label = if_else(col_a > 10, "high", "low"))`

If it breaks: `must be a logical vector, not the string "long"` means the three parts are out of order. The test comes first, then the true value, then the false value.

`transmute()`: keep only the columns you build

Like `mutate()`, but it drops every column you do not name. Use it to carve a wide table down to the few columns a test needs.

```
msgs %>%
  transmute(channel, words = message_length / 5) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  channel    words
  <chr>      <dbl>
1 sodapoppin   4.6
2 xqcow       88.2
3 forsen       1.6
4 sodapoppin   2.4
5 xqcow       41.2
```

On your data: `your_data %>% transmute(id, score = col_a + col_b)`

If it breaks: a column you expected is gone. `transmute()` keeps only what you list; name the ones you want to carry through, as `channel` is named above.

Count and summarize

`count()`: how many rows in each group

Counts the rows for each value of a column. Add `sort = TRUE` to put the biggest group first. Notice `count()` shows `NA` as its own group, which is a quick way to spot missing values.

```
msgs %>%
  count(channel_type)
```

```
# A tibble: 3 × 2
  channel_type     n
  <fct>        <int>
1 gaming       31309
2 non-gaming    3457
3 <NA>           501
```

On your data: `your_data %>% count(col_a)`, or `count(col_a, sort = TRUE)`

If it breaks: nothing to break. An unexpected `NA` row is telling you some rows have no value for that column.

`group_by()` plus `summarize()`: one row per group

`group_by()` sets the grouping column; `summarize()` then collapses each group to one row of numbers you ask for. `n()` counts the rows in the group. End with `ungroup()` so later steps see a plain table. Filter out `NA` groups first, and pass `na.rm = TRUE` so blanks do not turn a mean into `NA`.

```
msgs %>%
  filter(!is.na(channel_type)) %>%
  group_by(channel_type) %>%
  summarize(n = n(), mean_len = mean(message_length, na.rm = TRUE)) %>%
  ungroup()
```

```
# A tibble: 2 × 3
  channel_type     n mean_len
  <fct>        <int>    <dbl>
1 gaming       31309     28.5
2 non-gaming    3457     33.7
```

On your data: `your_data %>% group_by(col_a) %>% summarize(mean = mean(col_b, na.rm = TRUE))`

If it breaks: a mean of `NA` means a blank slipped in; add `na.rm = TRUE`. Numbers that look too much alike across groups usually mean you forgot `group_by()`.

`distinct()` and `n_distinct()`: count the unique values

`distinct()` keeps one row per unique value; `n_distinct()` counts them inside a `summarize()`.

```
msgs %>%
  distinct(channel) %>%
  nrow()
```

```
[1] 50
```

```
msgs %>%
  summarize(channels = n_distinct(channel))
```

```
# A tibble: 1 × 1
  channels
     <int>
1       50
```

On your data: `your_data %>% distinct(col_a)` or `summarize(k = n_distinct(col_a))`

If it breaks: nothing to break; a count higher than you expect usually means stray spaces or capitals making "Bob" and "bob" count twice.

`pull()`: take one column out as a plain vector

Pulls a single column out of a table as a bare vector, for when a later step wants values, not a one-column table.

```
msgs %>%
  distinct(viewer_band) %>%
  pull(viewer_band)
```

```
[1] large  medium small
Levels: small medium large
```

On your data: `your_data %>% pull(col_a)`

If it breaks: `pull()` with no column name returns the last column. Name the column to be sure which one you get.

Join tables

`left_join()`: add another table's columns by a shared key

Adds the columns of a second table onto the first, matching on a shared key column. Every row of the first table is kept; where no match is found, the new columns are `NA`.

```
lookup <- channels %>%
  select(channel, main_game)
msgs %>%
  left_join(lookup, by = "channel") %>%
  select(channel, main_game, message_length) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 3
  channel    main_game                   message_length
  <chr>      <chr>                                <int>
1 sodapoppin Hand Simulator                          23
2 xqcow      Just Chatting                          441
3 forsen     The Elder Scrolls V: Skyrim              8
4 sodapoppin Hand Simulator                          12
5 xqcow      Just Chatting                          206
```

On your data: `your_data %>% left_join(lookup, by = "id")`

If it breaks: if the row count jumps after a join, the key matched more than one row in the second table. Run `nrow()` before and after; the two must be equal. (`inner_join()` is the same call but drops rows with no match instead of keeping them as `NA`.)

`join_by()` with `closest()`: match on the nearest earlier time

For a join on something other than an exact match. `closest(ts >= snapshot_ts)` pairs each message with the most recent stream snapshot at or before it, which is how the wrangling worksheet attaches a viewer count to every message.

```
chat_bits <- chat %>%
  filter(channel == "bobross") %>%
  select(channel, ts, message_length) %>%
  slice_head(n = 3)
snaps <- streams %>%
  filter(channel == "bobross") %>%
  transmute(channel, snapshot_ts = ts, viewers) %>%
  slice_head(n = 3)
chat_bits %>%
  left_join(snaps, join_by(channel, closest(ts >= snapshot_ts)))
```

```
# A tibble: 3 × 5
  channel ts                  message_length snapshot_ts         viewers
  <chr>   <dttm>                       <int> <dttm>                 <int>
1 bobross 2018-11-18 22:11:53             76 2018-11-18 22:04:49    2698
```

```
2 bobross 2018-11-18 22:19:08              8 2018-11-18 22:04:49    2698
3 bobross 2018-11-18 22:24:14             27 2018-11-18 22:04:49    2698
```

On your data: `a %>% left_join(b, join_by(id, closest(time >= start)))`

If it breaks: `` `>=` is not a valid join condition `` usually means the two time columns are not both real date-times. Convert them with `as_datetime()` first (see the lubridate sheet).

`anti_join()`: find the rows that did not match

Returns the rows of the first table that have no match in the second. The worksheet uses it to catch codes that did not land on a message: it should return zero rows.

```
codes <- tibble(channel = c("bobross", "not_a_channel"))
codes %>%
  anti_join(msgs, by = "channel")
```

```
# A tibble: 1 × 1
  channel
  <chr>
1 not_a_channel
```

On your data: `your_codes %>% anti_join(your_data, by = "id")`

If it breaks: a row you expected to match shows up here. Its key differs from the key in the other table, often a changed `id` or a stray space.

Reshape and recode

`factor()`: store a category in the order you choose

Turns text into a factor, a category with its values held in the order you set, so `small` sorts before `medium` and `large` instead of alphabetically.

```
bands <- tibble(viewer_band = c("large", "small", "medium", "small"))
bands %>%
  mutate(viewer_band = factor(viewer_band,
                              levels = c("small", "medium", "large"))) %>%
  arrange(viewer_band)
```

```
# A tibble: 4 × 1
  viewer_band
  <fct>
1 small
2 small
```

```
3 medium
4 large
```

On your data: `mutate(col = factor(col, levels = c("low", "mid", "high")))`

If it breaks: a value becomes `NA` after `factor()` when it is misspelled in `levels`. Every value in the column must appear in the `levels` list exactly.

`fct_reorder()`: order a category by a number

From **forcats**. Sets a factor's order by another column's values, so a bar chart draws its bars shortest to tallest instead of alphabetically.

```
by_type <- msgs %>%
  filter(!is.na(channel_type)) %>%
  group_by(channel_type) %>%
  summarize(mean_len = mean(message_length, na.rm = TRUE)) %>%
  mutate(channel_type = forcats::fct_reorder(channel_type, mean_len))
levels(by_type$channel_type)
```

```
[1] "gaming"     "non-gaming"
```

On your data: `mutate(group = fct_reorder(group, score))`

If it breaks: `could not find function "fct_reorder"` means forcats is not attached; it comes with `library(tidyverse)`.

`cut()`: turn a number into bands

Splits a number into labeled bands at the break points you give. The breaks run from one edge to the next; `Inf` means "and everything above."

```
tibble(viewers = c(3, 847, 5200, 21548)) %>%
  mutate(band = cut(viewers,
                    breaks = c(0, 1000, 10000, Inf),
                    labels = c("small", "medium", "large")))
```

```
# A tibble: 4 × 2
  viewers band
    <dbl> <fct>
1       3 small
2     847 small
3    5200 medium
4   21548 large
```

On your data: `mutate(band = cut(col, breaks = c(0, 10, 100, Inf), labels = c("a","b","c")))`

If it breaks: you need one more break than you have labels. Three labels need four break points, as above.

`pivot_wider()`: spread one column into many

Turns the values of one column into their own columns, so a long list of counts becomes a table you can read across. `values_fill = 0` writes `0` where a combination never occurred.

```
msgs %>%
  filter(!is.na(channel_type)) %>%
  count(viewer_band, channel_type) %>%
  pivot_wider(names_from = channel_type, values_from = n, values_fill = 0)
```

```
# A tibble: 3 × 3
  viewer_band gaming `non-gaming`
  <fct>        <int>        <int>
1 small        25309         2457
2 medium        4000         1000
3 large         2000            0
```

On your data: `your_counts %>% pivot_wider(names_from = col_a, values_from = n)`

If it breaks: `Values in `n` are not uniquely identified` means the rows are not counted yet. Run `count()` first so each combination appears once.

`pivot_longer()`: stack many columns into one

The reverse of `pivot_wider()`: folds several columns back into one name column and one value column. Charts usually want the long shape.

```
wide <- msgs %>%
  filter(!is.na(channel_type)) %>%
  count(viewer_band, channel_type) %>%
  pivot_wider(names_from = channel_type, values_from = n, values_fill = 0)
wide %>%
  pivot_longer(cols = c(gaming, `non-gaming`),
               names_to = "channel_type", values_to = "n")
```

```
# A tibble: 6 × 3
  viewer_band channel_type     n
  <fct>       <chr>        <int>
1 small       gaming       25309
```

```
2 small       non-gaming    2457
3 medium      gaming        4000
4 medium      non-gaming    1000
5 large       gaming        2000
6 large       non-gaming       0
```

On your data: `your_wide %>% pivot_longer(cols = c(col_a, col_b), names_to = "key", values_to = "value")`

If it breaks: a column name with a space, such as `non-gaming`, needs backticks around it in the `cols` list, as shown.

`drop_na()`: remove rows with a blank

Drops every row that has `NA` in the column you name.

```
tibble(channel = c("a", "b", "c"), viewers = c(10, NA, 30)) %>%
  drop_na(viewers)
```

```
# A tibble: 2 × 2
  channel viewers
  <chr>     <dbl>
1 a            10
2 c            30
```

On your data: `your_data %>% drop_na(col_a)`

If it breaks: `drop_na()` with no column name drops a row if *any* column is blank, which can remove more than you meant. Name the column you care about.

Also written by the R Companion

These turn up in the code the R Companion writes for the describing and inferencing steps. You do not usually type them by hand, but you should be able to read them.

`across()`: do the same thing to several columns

Applies one calculation to several columns at once, inside `summarize()` or `mutate()`.

```
msgs %>%
  filter(!is.na(channel_type)) %>%
  group_by(channel_type) %>%
  summarize(across(c(message_length, viewers), ~ mean(.x, na.rm = TRUE)))
```

```
# A tibble: 2 × 3
  channel_type message_length viewers
```

```
  <fct>                <dbl>   <dbl>
1 gaming                28.5   5054.
2 non-gaming            33.7   6773.
```

`starts_with()`: choose columns by the start of their name

A helper for `select()` and `rename()` that matches every column whose name begins with the text you give.

```
msgs %>%
  select(starts_with("channel")) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  channel    channel_type
  <chr>      <fct>
1 sodapoppin gaming
2 xqcow      non-gaming
3 forsen     gaming
4 sodapoppin gaming
5 xqcow      non-gaming
```

`bind_cols()`: set two tables side by side

Glues two tables together column-wise. They must have the same number of rows, in the same order.

```
a <- tibble(channel = c("bobross", "xqcow"))
b <- tibble(viewers = c(3210, 19381))
bind_cols(a, b)
```

```
# A tibble: 2 × 2
  channel viewers
  <chr>     <dbl>
1 bobross    3210
2 xqcow     19381
```

`slice()`: keep rows by position

Keeps the rows at the positions you name. `slice(1)` keeps the first row.

```
msgs %>%
  slice(1)
```

```
# A tibble: 1 × 6
  channel    channel_type viewer_band viewers message_length  hour
  <chr>      <fct>        <fct>         <int>          <int> <int>
1 sodapoppin gaming       large            NA             23    21
```

`last()`: the last value in a column

Returns the last value of a vector, used inside `summarize()`.

```
msgs %>%
  summarize(last_hour = last(hour))
```

```
# A tibble: 1 × 1
  last_hour
      <int>
1        18
```